</> Shell Scripting and Networking Tools

05-03-2026

☑ Pre-Session Checklist

Before we dive in, make sure your arsenal is stocked. Run this quick check:

```
# Check if essential packages are installed
$ which curl aria2c ssh wget ping ip

# If anything is missing, install them:
# Debian/Ubuntu: sudo apt install curl aria2 openssh-client wget
iproute2
# Arch: sudo pacman -S curl aria2 openssh wget iproute2
# Fedora: sudo dnf install curl aria2 openssh-clients wget iproute2
```

Part 1: Bash Scripting — Beyond the Basics

You already know how to navigate the terminal. You've used `grep`, `sed`, and `awk`. You've redirected output and written basic scripts. Now it's time to think like a developer and build tools that actually *do* things.

> **Video Reference:** For the foundational concepts, <Click Here>. Remember: we covered `Basic commands`, `Output/Input redirection`, `AWK`, and `SED` in previous sessions, so we'll touch on them only briefly here.

1.1 Variables and Data Types

In Bash, everything is a string. But that doesn't mean you can't be clever.

```
#!/bin/bash

# Variable assignment (no spaces!)
user_name="hacker"
age=21
is_cool=true

# Accessing variables
echo "Hey $user_name, you're $age years old!"

# Command substitution
current_date=$(date +%Y-%m-%d)
kernel_version=$(uname -r)

echo "Today is $current_date, running kernel $kernel_version"
```

Pro Tip: Use `$()` for command substitution instead of backticks. It's nestable and easier to read.

1.2 Control Flow

If Statements

```
#!/bin/bash

file_count=$(ls | wc -l)

if [ $file_count -gt 10 ]; then
    echo "Directory is cluttered ($file_count files). Time to clean up!"
elif [ $file_count -eq 0 ]; then
    echo "Empty directory. Lonely vibes."
else
    echo "Looking good with $file_count files."
fi
```

Important: Always use spaces inside brackets `[ ]`. `[ $var -eq 1 ]` works, `[$var -eq 1]` fails.

Case Statements

Perfect for handling multiple options cleanly:

```
#!/bin/bash

echo "Choose your distro:"
echo "1) Arch"
echo "2) Debian"
echo "3) Fedora"
read choice

case $choice in
    1)
        echo "I use Arch, btw"
        ;;
    2)
        echo "Stable and reliable. Nice."
        ;;
    3)
        echo "Red Hat family represent!"
        ;;
    *)
        echo "Invalid choice. Try again."
        ;;
esac
```

1.3 Loops

For Loops

```
#!/bin/bash

# Iterate over files
for file in *.txt; do
    echo "Processing $file..."
    wc -l "$file"
done

# C-style for loop
for ((i=0; i<5; i++)); do
    echo "Iteration $i"
done
```

While Loops

```
#!/bin/bash

# Read file line by line
while IFS= read -r line; do
    echo "Line: $line"
done < input.txt

# Infinite loop with break
counter=0
while true; do
    echo "Loop $counter"
    ((counter++))
    [ $counter -eq 5 ] && break
done
```

1.4 Functions

Write reusable code blocks:

```
#!/bin/bash

# Function definition
check_port() {
    local host=$1
    local port=$2
    timeout 2 bash -c "</dev/tcp/$host/$port" 2>/dev/null && \
        echo "Port $port on $host is OPEN" || \
        echo "Port $port on $host is CLOSED"
}

# Call functions
check_port "google.com" 443
check_port "localhost" 22
```

Key Points:

- Use `local` for variables inside functions to avoid global scope pollution
- Functions should do one thing well (Unix philosophy)
- Return values are 0-255 (exit codes), not strings

1.5 Error Handling

Don't let your scripts crash silently:

```
#!/bin/bash

# You Can Use Automatic Error Handling
set -euo pipefail
# -e: Exit on error
# -u: Exit on undefined variable
# -o pipefail: Catch errors in pipelines

# Or handle errors manually
if ! ping -c 1 google.com &> /dev/null; then
    echo "ERROR: No internet connection" >&2
    exit 1
fi
```

1.6 Working with APIs

Modern scripting is all about data. Here's how to fetch and parse JSON:

⚠ Check that `jq` and `curl` are installed before running this script below.

```
#!/bin/bash

# Fetch data from API
response=$(curl -s "https://api.github.com/users/torvalds")

# Parse with jq (install it!)
login=$(echo "$response" | jq -r '.login')
followers=$(echo "$response" | jq -r '.followers')
created_at=$(echo "$response" | jq -r '.created_at')

echo "User: $login"
echo "Followers: $followers"
echo "Joined: $created_at"
```

Quick Reference: I/O Redirection

(Review from previous sessions)

Operator	Description
>	Redirect stdout to file (overwrite)
>>	Redirect stdout to file (append)
<	Redirect file to stdin
2>	Redirect stderr
&>	Redirect both stdout and stderr
|	Pipe stdout to another command
2>&1	Redirect stderr to stdout

Quick Reference: Text Processing

(Review from previous sessions)
AWK - Column extraction and calculations

```
$ awk '{print $1}' file.txt          # Print first column
$ awk -F: '{print $NF}' /etc/passwd  # Print last field (delimiter :)
$ awk '{sum+=$1} END {print sum}'    # Sum first column
```

SED - Stream editing

```
$ sed 's/old/new/g' file.txt         # Replace all occurrences
$ sed -i 's/foo/bar/' file.txt       # Edit in-place
$ sed -n '5,10p' file.txt            # Print lines 5-10 only
```

Part 2: Networking Tools — The Hacker's Swiss Army Knife

Linux networking tools are your eyes and ears into the digital world. Whether you're debugging connectivity, downloading files, or managing remote systems, these commands are essential.

2.1 Connectivity Testing

ping — The Classic

Official Documentation: `man ping`

```
# Basic usage
$ ping google.com

# Limit count (stop after 4 packets)
$ ping -c 4 google.com

# Set interval (faster ping, requires sudo)
$ sudo ping -i 0.2 google.com

# Audible ping (beep on reply)
$ ping -a 192.168.1.1
```

What it does: Sends ICMP echo requests to test host reachability and measure latency.

traceroute / tracepath

```
# See the path your packets take
$ traceroute google.com

# Alternative without root (uses UDP instead of ICMP)
$ tracepath google.com
```

2.2 Network Interface Configuration

ip (Modern Replacement for ifconfig)

Official Documentation: `man ip`

```
# Show all interfaces
$ ip addr show
$ ip a  # Short form

# Show specific interface
$ ip addr show eth0
```

```
# Bring interface up/down
$ sudo ip link set eth0 up
$ sudo ip link set eth0 down

# Add IP address
$ sudo ip addr add 192.168.1.100/24 dev eth0

# Show routing table
$ ip route show

# Add route
$ sudo ip route add default via 192.168.1.1
```

Why `ip` over `ifconfig`? `ip` is part of the iproute2 package, actively maintained, and supports modern networking features like network namespaces and policy routing.

ifconfig (Legacy)

Official Documentation: `man ifconfig`

```
# Still works on many systems
$ ifconfig
$ ifconfig eth0
$ sudo ifconfig eth0 up
```

Note: While `ifconfig` is deprecated, you'll still encounter it on older systems and embedded devices.

2.3 Data Transfer

curl — The URL Swiss Army Knife

Official Documentation: `man curl`, curl.se/docs

```
# Simple GET request
$ curl https://api.example.com/data
```

```
# Save to file
$ curl -o file.zip https://example.com/file.zip

# Follow redirects
$ curl -L https://bit.ly/xyz

# Include headers
$ curl -I https://google.com

# POST data
$ curl -X POST -d "name=john&age=30" https://api.example.com/users

# POST JSON
$ curl -X POST \
  -H "Content-Type: application/json" \
  -d '{"user":"john","pass":"secret"}' \
  https://api.example.com/login

# Download with resume capability
$ curl -C - -o large.iso https://example.com/large.iso

# Use authentication
$ curl -u username:password https://api.example.com/protected

# Silent mode (progress bar only)
$ curl -s https://api.ipify.org

# Show progress and speed
$ curl --progress-bar -o file.zip https://example.com/file.zip
```

Pro Tip: Create a `.curlrc` file in your home directory for default options like `-L` (follow redirects) or custom headers.

wget — The Download Specialist

Official Documentation: `man wget`, GNU Wget Manual

```
# Simple download
$ wget https://example.com/file.zip
```

```
# Download to specific directory
$ wget -P ~/Downloads https://example.com/file.zip

# Resume broken download
$ wget -c https://example.com/large.iso

# Download entire website (spider mode)
$ wget --spider https://example.com

# Mirror website locally
$ wget --mirror --convert-links --adjust-extension --page-requisites \
  --no-parent https://example.com/docs

# Download in background
$ wget -b https://example.com/large-file.tar.gz

# Limit download speed (be nice to servers!)
$ wget --limit-rate=500k https://example.com/file.zip

# Download via FTP
$ wget ftp://user:pass@ftp.example.com/file.txt

# Recursive download with specific file types
$ wget -r -A.pdf https://example.com/documents/
```

⚖ **curl vs wget:** Use `curl` for APIs and complex HTTP operations. Use `wget` for reliable downloads, mirroring, and recursive fetching.

aria2 — The Download Accelerator

📖 **Official Documentation:** `man aria2c`, aria2.github.io

```
# Multi-connection download (faster!)
$ aria2c -x 16 -s 16 https://example.com/large.iso

# Download BitTorrent
$ aria2c https://example.com/file.torrent

# Download with Metalink
$ aria2c file.metalink
```

```
# Download multiple files
$ aria2c -i urls.txt

# Limit speed
$ aria2c --max-download-limit=2M https://example.com/file.zip

# Continue incomplete download
$ aria2c -c https://example.com/large.iso

# RPC mode (for web UI control)
$ aria2c --enable-rpc --rpc-listen-all
```

Why aria2? It splits files into multiple segments and downloads them simultaneously, often saturating your bandwidth better than single-threaded tools.

2.4 Remote Access

ssh — Secure Shell

Official Documentation: `man ssh`, OpenSSH Manual

```
# Basic connection
$ ssh username@remote-host

# Specify port
$ ssh -p 2222 username@remote-host

# Execute command remotely
$ ssh user@host "ls -la /var/log"

# Tunnel local port to remote
$ ssh -L 8080:localhost:80 user@host

# Reverse tunnel (expose local port on remote)
$ ssh -R 9090:localhost:3000 user@host

# SOCKS proxy (browse securely through remote)
$ ssh -D 1080 user@host

# X11 forwarding (run GUI apps remotely)
```

```
$ ssh -X user@host

# Key-based authentication (no password!)
$ ssh-copy-id user@host

# Use specific key
$ ssh -i ~/.ssh/my_key user@host

# Config file (~/.ssh/config)
Host myserver
    HostName 192.168.1.100
    User admin
    Port 2222
    IdentityFile ~/.ssh/my_key
```

Next Steps:

- Disable password authentication in `/etc/ssh/sshd_config`
- Use key pairs with passphrase
- Change default port (security through obscurity + fewer log spam)
- Use `fail2ban` to block brute force attempts

scp — Secure Copy

Official Documentation: `man scp`

```
# Copy local to remote
$ scp file.txt user@host:/home/user/

# Copy remote to local
$ scp user@host:/var/log/syslog ./local-syslog

# Copy directory recursively
$ scp -r ./project user@host:/var/www/

# Preserve file attributes
$ scp -p file.txt user@host:/backup/

# Limit bandwidth (KB/s)
```

```
$ scp -l 1000 large.iso user@host:/tmp/

# Use specific SSH port
$ scp -P 2222 file.txt user@host:/home/user/
```

Modern Alternative: `rsync` is often preferred for large transfers as it supports resume and delta transfer.

Part 3: Ricing Linux — Making It Yours

Why Ricing Matters

Look, you could use Windows or macOS and accept whatever design choices some corporation made for you. Or... you could build a computing environment that matches *your* workflow, *your* aesthetic, and *your* personality.

Ricing is the art of customizing your Linux desktop. The term comes from "Rice Rocket" — cheap cars modified to look expensive and perform better. In the Linux world, it's about taking a minimal base and making it beautiful and functional.

The Revelation: <Click Here> to watch the video . Seriously, this is the video that inspired this entire course. It shows what's possible when you stop accepting defaults and start crafting your digital workspace.

Why It Matters for You?

1. **Productivity:** Tiling window managers let you navigate entirely with keyboard shortcuts. No more hunting for that tiny close button.
2. **Minimalism:** Your 10-years-old laptop can feel brand new when you're not running bloated desktop environments.
3. **Aesthetics:** Your desktop can look like it belongs in a cyberpunk movie. Neon colors, transparency, animated wallpapers — it's art.
4. **Learning:** Configuring a tiling WM teaches you more about Linux than any tutorial ever could.
5. **Community:** Share your setup, get feedback and inspire others.

The Components of a Riced Setup

Window Managers vs Desktop Environments

Feature	Desktop Environment	Window Manager
Resource Usage	High (500MB-1GB RAM)	Low (10-50MB RAM)
Customization	Limited/themed	Unlimited
Learning Curve	Gentle	Steep but rewarding
Built-in Tools	File manager, panel, settings	None (you choose everything)
Aesthetic	Corporate/polished	Minimal/personal

Popular Tiling Window Managers

- **i3/i3-gaps:** The gateway drug. Easy config, great docs, huge community.
- **bspwm:** Binary space partitioning. Scriptable with bash.
- **AwesomeWM:** Built on Lua, very powerful.
- **Hyprland:** Modern Wayland compositor with eye candy animations.
- **Qtile:** Python-based, hackable, dynamic.

Essential Ricing Components

1. **Status Bar:** Display workspace info, system stats, time. Try **Polybar** (X11) or **Waybar** (Wayland), or you can make a simple script to do this for you.
2. **Application Launcher:** Ditch the menu. Use **rofi** or **dmenu** for lightning-fast app launching.
3. **Terminal:** Your most-used app, choose what you are comfortable with.
4. **Shell:** Upgrade from Bash to **Zsh** (with Oh-My-Zsh) or **Fish** for better autocomplete and plugins.
5. **Compositor:** Add blur, transparency, animations. **picom** or **compton** for X11, built-in for Wayland.
6. **Dotfiles:** Store your configs on Github repo or any git based service. When your system dies, `git clone` and you're back.

Getting Started

Don't rice your main machine immediately. Use a VM or old laptop. Choose a WM that you have liked. Read its User's Guide, modify the config line by line, and watch your system become uniquely *yours*.

Remember: ricing is never ”done.” It’s a continuous process of tweaking, breaking, fixing, and discovering. Your setup evolves with you.

Homework for Session 11

Time to switch from theory to practice !!

Assignment 1: Info Script

Write a bash script that uses `curl` to fetch and display one of the following:

1. **Currency rates** (USD to EUR, USD to GBP, USD to JPY) — use exchangerate-api.com or similar
2. **Prayer times** for your city — use aladhan.com API
3. **Weather forecast** — use OpenWeatherMap or wttr.in

Suggestions:

- Clean, colored output
- Error handling for failed API calls (maybe the script runs without internet connection)
- Configurable (city, currency base stored as variables)
- Use functions for each data type

Assignment 2: Mobile SSH Mastery

Install Termux (Android) or iSH (iOS) on your phone and:

1. Generate SSH keys on your mobile device
2. Copy the public key to your PC’s ‘`~/.ssh/authorized_keys`‘
3. Successfully SSH into your PC from your phone over local WiFi
4. Run a command on your PC from your phone (take a screenshot, check system stats, etc.)

Course Final Project

Congratulations!! You've made it through the course. Now it's time to practice what you've learned.

Suggested Practice: Build Your Setup

At home, complete the following:

1. **Install Linux**
   - Any distribution you choose (Arch, Debian, Fedora, etc.)
   - Bare metal or VM (bare metal preferred for full experience)
2. **Rice Your Desktop**
   - Create a customized, personalized desktop environment
   - Desktop Environments allowed: GNOME (heavily themed), KDE Plasma, XFCE
   - Window Managers allowed: i3, bspwm, Awesome, Hyprland, Qtile, Sway, or any other
3. **Document Your Journey**
   - Take screenshots of your progress
   - Keep your dotfiles in a Git repository
   - Write a short README explaining your choices
4. **Share With The Group**
   - Post final screenshots to the group
   - Share your dotfiles repository link
   - Explain one challenging part and how you solved it

Best Practices:

- Functionality (does it work for daily use?)
- Aesthetics (is it visually cohesive?)
- Technical complexity (did you challenge yourself?)
- Documentation (can others learn from your setup?)

Resources & Further Reading

- **Bash Guide:** BashGuide
- **ShellCheck:** shellcheck.net — Lint your scripts
- **ExplainShell:** explainshell.com — Understand complex commands
- **Arch Wiki:** wiki.archlinux.org — Rich Linux documentation

End of Linux Course

Now go forth and script, network, and rice like the Linux wizard you're becoming.

References

- ping(8) - Linux man page - https://linux.die.net/man/8/ping
- ip(8) - Linux man page - https://man7.org/linux/man-pages/man8/ip.8.html
- ifconfig(8) - Linux man page - https://linux.die.net/man/8/ifconfig
- curl documentation - https://curl.se/docs/
- GNU Wget Manual - https://www.gnu.org/software/wget/manual/wget.html
- aria2 documentation - https://aria2.github.io/manual/en/html/
- OpenSSH Manual - https://www.openssh.com/manual.html
- scp(1) - Linux man page - https://man7.org/linux/man-pages/man1/scp.1.html

تمت بحمد الله